Exercise — Endpoints

version **#1.0.0**

Yet Another Kind of Assistants 2027 <assistants@tickets.assistants.epita.fr>

Copyright

This document is for internal use at EPITA (website) only.

Contents

*https://intra.forge.epita.fr

File Tree


endpoints/
pom.xml
src/
main/
java/
fr/
epita/
assistants/
presentation/
rest/
Endpoints.java (to submit)
HelloWorldEndpoint.java
request/
ReverseRequest.java (to submit)
response/
HelloResponse.java (to submit)
ReverseResponse.java (to submit)
resources/
application.properties
openapi.yaml

Obligations

*Obligations are **fundamental** rules shared by all subjects. They are non-negotiable and to not apply them means to face sanctions. Therefore, do not hesitate to ask for explanations if you do not understand one of these rules.*

Obligation #0: **Cheating**, as well as sharing source code, tests, test tools or coding-style correction tools is **strictly forbidden** and penalized by not being graded, being flagged as a cheater and reported to the academic staff.

Obligation #1: Your submission repository must be **clean**. Except for special cases, which (if any) are **explicitely** mentioned in this document, an *unclean* repository may contain:

- binary files;[1]
- files with inappropriate privileges;
- forbidden files: `*~`, `*.swp`, `*.o`, `*.a`, `*.so`, `*.class`, `*.log`, `*.core`, etc.;
- a file tree that does not follow our specifications.

[1] If an executable file is required, please provide its sources **only**. We will compile it ourselves.

1 Objectives

The goal of this exercise is to create two simple endpoints: hello and reverse.

hello
: Should be a GET endpoint with a path parameter

reverse
: Should be a POST endpoint with no path parameter

For this exercise, you won't need to implement any service and therefore will only create a `presentation` layer. You will need to follow the package architecture shown above.

Tips

The specifications of the endpoints can be found in the Swagger file *src/main/resources/openapi.yaml*. Every behavior described in the Swagger documentation, including error handling, must be implemented. We **strongly** advise you to use it.

2 Given rest API annotations

Here is a list of annotations that you will use to create your REST API.

- `@Path`: This annotation defines the URI for a method.
- `@GET`: This annotation specifies that the following method is a GET method.
- `@POST`: This annotation specifies that the following method is a POST method.
- `@PATCH`: This annotation specifies that the following method is a PATCH method.
- `@Produces` / `@Consumes`: Those annotations define the type of data, which correspond to the mediatype, that the method will respectively return or accept as input.
- `@PathParam`: This annotation allows us to retrieve an attribute of the request.

Finally, you should take a look at the Response class from the jakarta.ws.rs.core package for the responses sent by your endpoints.

Tips

- This subject is voluntarily short. You are encouraged to explore the Quarkus documentation and thoroughly read the provided swagger file to guide you on how your endpoints should behave. You may find the guide "Simplified Hibernate ORM with Panache" to be particularly useful.
- This exercise is fundamental to understanding how to start the JWS project. After completing it, make sure you understand everything you have done before moving on.
- Here is a video that explains everything you need to know about the Rest API.

3 Given Files

In order to start the project in good conditions, a tarball is available on the intranet. It contains all the files needed for the proper functioning of the project.

You **should** decompress this tarball and open the directory with IntelliJ before reading the rest of the subject.

We provided you with an example on how to create an endpoint in the *HelloWorldEndpoint.java* file.

To see what the provided example will give you, you should use the following command:

Tips

See the next section for more information on how to start the project before trying the following command.

```
42sh$ curl http://localhost:8082/
```

4 How To Start

When the project is open in your IDE, you can type the following command to start the server.

```
42sh$ mvn quarkus:dev
```

It will launch your API on the port 8082. In order to make a request on your endpoints you should use the following commands:

```
42sh$ curl http://localhost:8082/hello/{name} # replace {name} with an actual value
42sh$ curl http://localhost:8082/reverse -X POST -H "Content-Type: application/json" \
    -d '{"content": "hello world"}'
```

This should produce:

```
{"content":"hello {name}"}
{"original":"hello world","reversed":"dlrow olleh"}
```

Knowledge is a paradox... The more one understands, the more one realizes the vastness of his ignorance.